

Oprogramowanie Systemowe

Procesy i wątki w Windows

Procesy i wątki w Windows

- wykonywanie programu rozpoczyna się od wątku głównego (ang. main thread), który w tradycyjnym programowaniu w języku C jest funkcją *main* (*WinMain* w systemie Windows);
- w trakcie wykonywania proces może tworzyć nowe wątki poprzezwołanie funkcji systemowej, której parametrem jest nazwa funkcji innego wątku;
- system operacyjny (w którym używana jest metoda wywłaszczania) przekazuje sterowanie wśród wątków tak samo jak przekazuje sterowanie wśród procesów
- poprawne projektowanie, kodowanie i uruchamianie programów wielowątkowych jest zadaniem trudnym; ponieważ przełączanie wątków może wystąpić w dowolnym momencie czasu, pewne niepożądane interakcje między wątkami mogą wystąpić tylko przypadkowo; m.in. konieczne jest stosowanie różnych form synchronizacji wątków.

Tworzenie procesów w Win

- funkcja `CreateProcess`, zdefiniowana w systemie Win32 API, tworzy nowy proces (i jego wątek główny);
- poniżej podano prototyp funkcji `CreateProcess`

```
BOOL CreateProcess(  
    LPCTSTR lpApplicationName,  
    LPTSTR lpCommandLine,  
    LPSECURITY_ATTRIBUTES lpProcessAttributes,  
    LPSECURITY_ATTRIBUTES lpThreadAttributes,  
    BOOL bInheritHandles,  
    DWORD dwCreationFlags,  
    LPVOID lpEnvironment,  
    LPCTSTR lpCurrentDirectory,  
    LPSTARTUPINFO lpStartupInfo,  
    LPPROCESS_INFORMATION  
    lpProcessInformation );
```

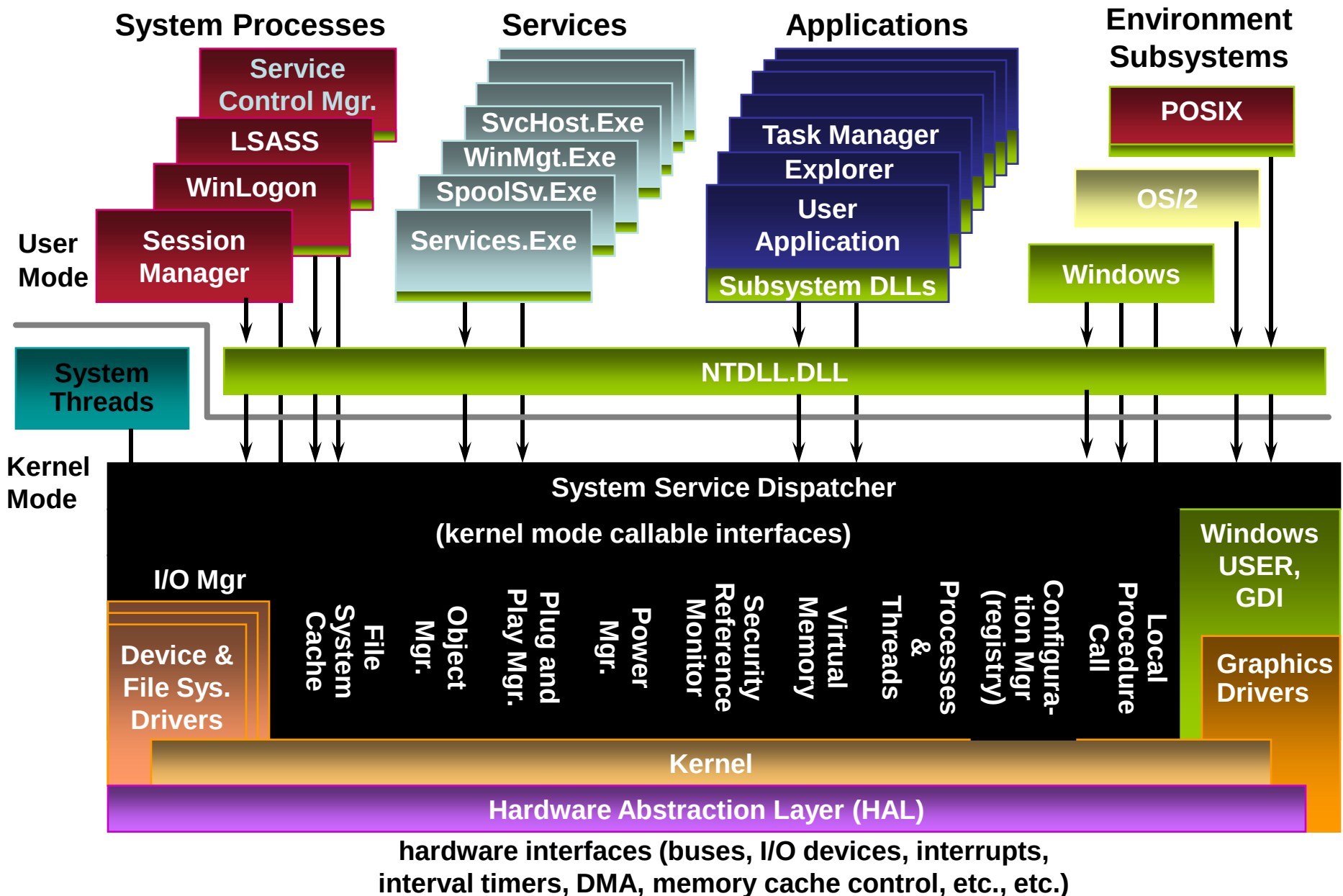
Wątki w Windows

- po wykonaniu funkcji tworzącej wątek, funkcja wątku może być wykonywana natychmiast w sposób współbieżny; wątek może być utworzony także jako *wstrzymany* (w stanie oczekiwania) — uruchomienie go będzie możliwe więc w dowolnym późniejszym momencie
- zasadniczo do tworzenia wątków używana jest funkcja `CreateThread`, ale w programach w języku C, które używają standardowej biblioteki funkcji, zalecane jest używanie funkcji `_beginthreadex`;
- planowanie w systemie Windows wykonywane jest na poziomie wątków, a nie procesów; przykładowo jeśli proces A ma 10 wątków gotowych do wykonania, a proces B ma 2 wątki, i wszystkie wątki mają jednakowe priorytety, to każdy z wątków otrzyma $\frac{1}{12}$ czasu procesora.

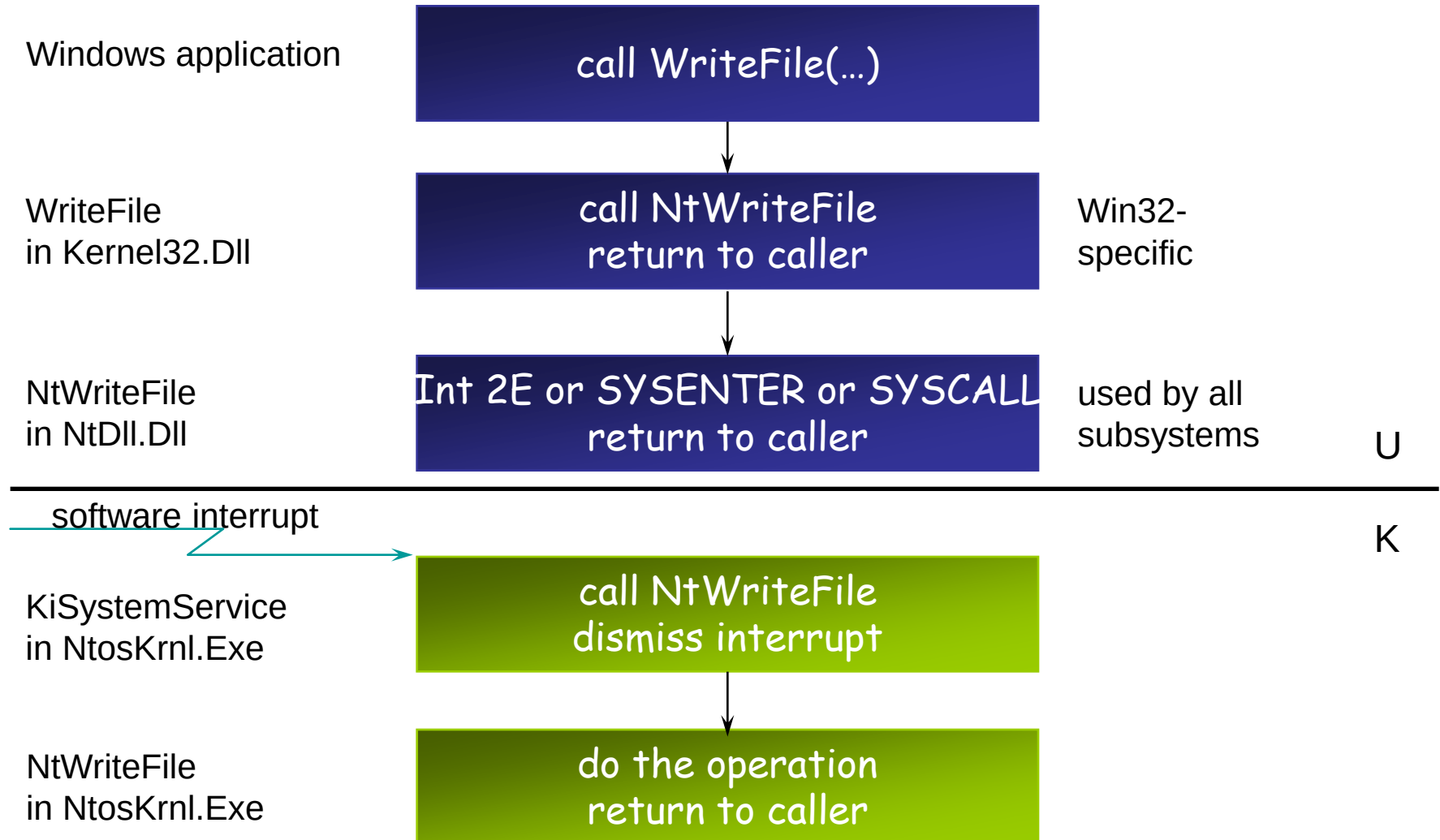
Funkcje oczekiwania

```
unsigned __stdcall FunkcjaWatku( void * arg) // funkcja watku
{
    char * znaki = (char*) arg;
    MessageBox(0,znaki,"Testowanie wątków",0);
    _endthreadex( 0 );
    return 0;
};
int main()
{
    unsigned IdentWatku[2];
    HANDLE hWatki[2];
    char w1[]="Pierwszy wątek";
    char w2[]="Drugi wątek";
    hWatki[0] = (HANDLE)_beginthreadex (NULL, 0, &FunkcjaWatku, (void *)w1, 0,
    &IdentWatku[0]);
    hWatki[1] = (HANDLE)_beginthreadex (NULL, 0, &FunkcjaWatku, (void *)w2, 0,
    &IdentWatku[1]);
    // oczekiwanie na zakończenie wykonywania wątków
    WaitForMultipleObjects(2, hWatki, TRUE, INFINITE );
    // zamknięcie dojsć (handle) wątków
    CloseHandle( hWatki[0] );
    CloseHandle( hWatki[1] );
    return 0;
}
```

Architektura Windows XP



Wywołanie funkcji systemowych kernela



Przykładowy kod

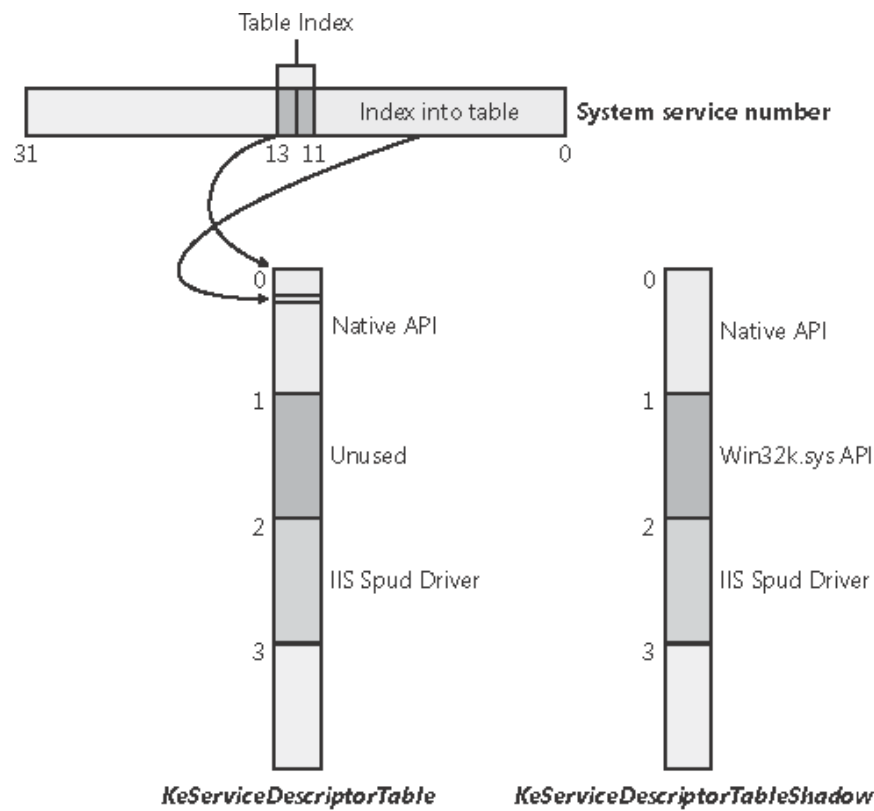
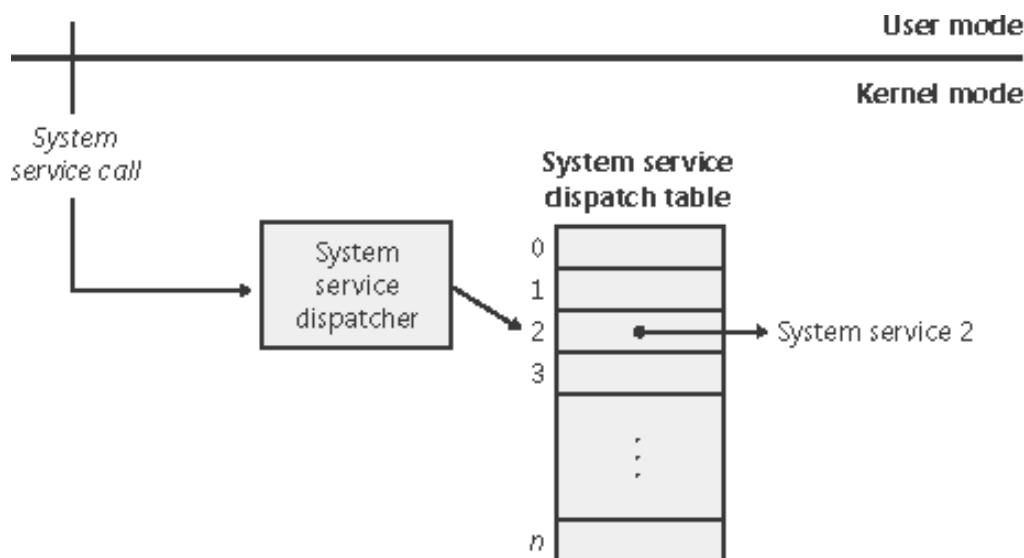
ntdll!NtReadFile:

77f5bfa8 b8b7000000	mov eax,0xb7
77f5bfad ba0003fe7f	mov edx,0x7ffe0300
77f5bfb2 ffd2	call edx
77f5bfb4 c22400	ret 0x24

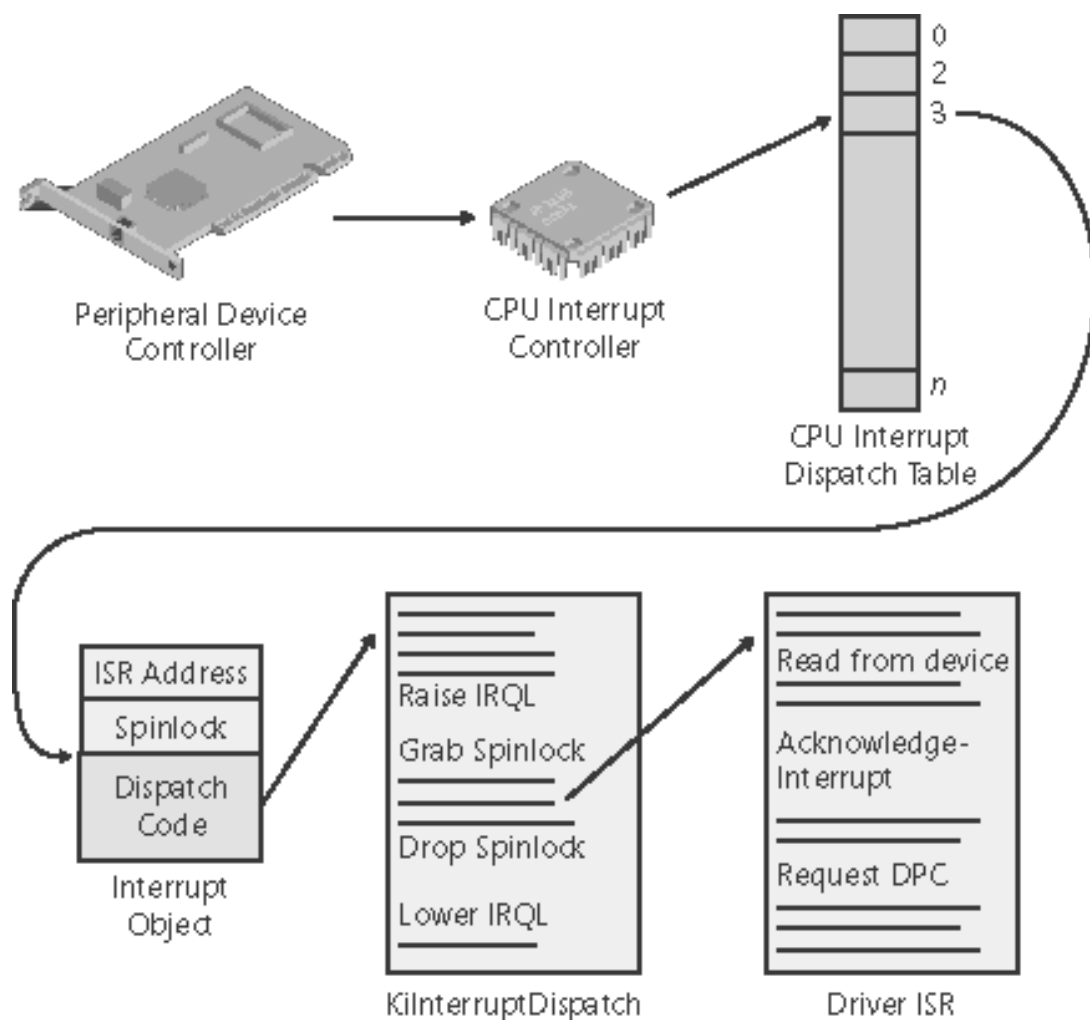
SharedUserData!SystemCallStub:

7ffe0300 8bd4	mov edx,esp
7ffe0302 0f34	sysenter
7ffe0304 c3	ret

Dalsza część przetwarzania



Przerwania



Wirująca blokada (Spinlock)

synchronizacja wątków na poziomie oprogramowania sprowadza się zwykle do cyklicznego testowania zawartości pewnej zmiennej współdzielonej przez kilka wątków, co określane jest terminem *wirująca blokada*,

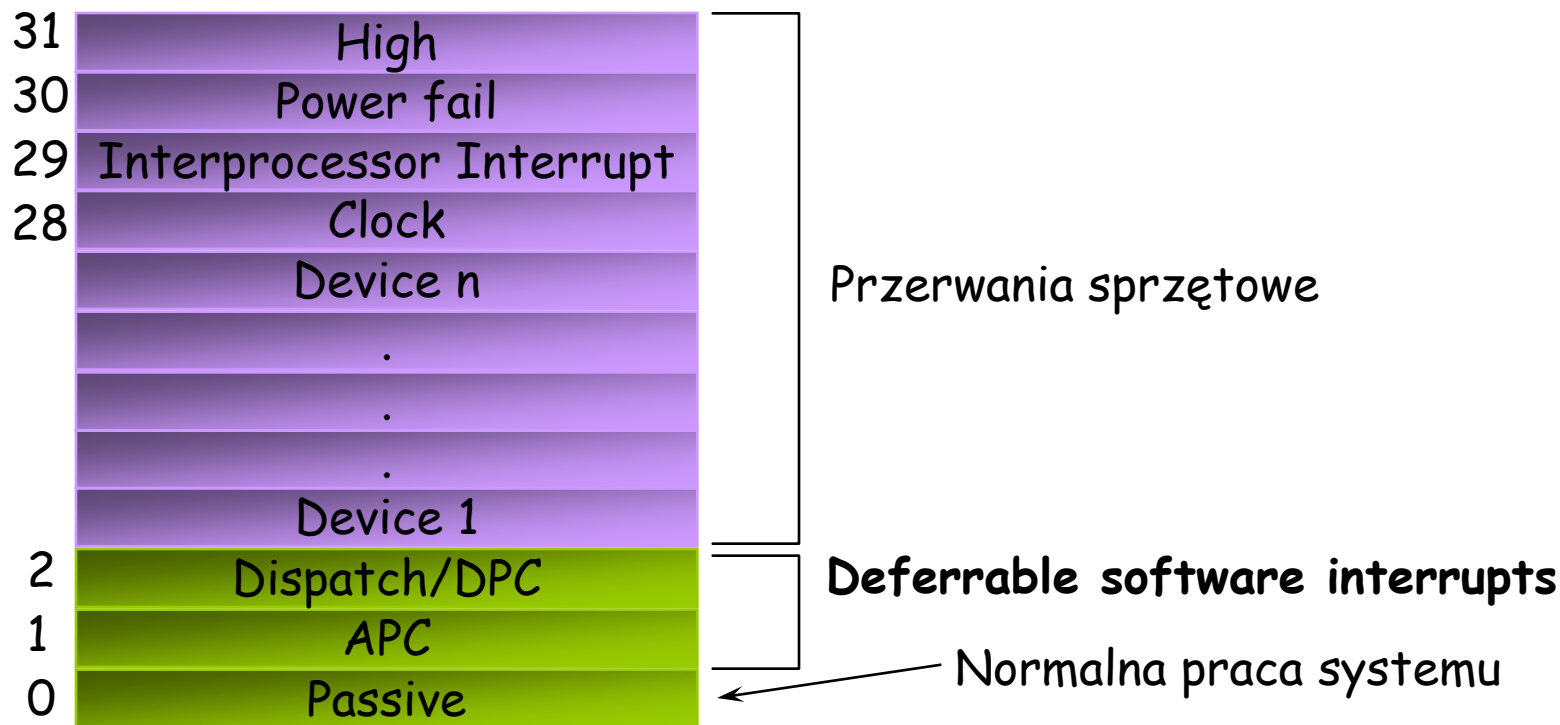
w architekturze IA-32 dostępny jest rozkaz XCHG, który zamienia zawartości podanych operandów; przypuśćmy, że zmienna zamek dostępna jest dla kilku wątków wykonywanych przez różne procesory;

```
zamek      db      0
_ _ _ _ _
powtorz:    mov     al, 1
            xchg    zamek, al
            cmp     al, 1
            je      powtorz
_ _ _ _ _ ; sekcja krytyczna (tylko jeden wątek
_ _ _ _ _ ; może znajdować się w tej sekcji)
            mov     zamek, 0
```

- warunkiem poprawnej realizacji podanego kodu jest niepodzielne wykonywanie rozkazu XCHG;

Poziomy uprzywilejowania przerwań IRQL

- IRQL = Interrupt Request Level
 - To nie to samo co IRQ (27-IRQ)
- IRQL jest stanem procesora (niejako wirtualnym)
- Nie wolno oczekiwać na obiekcie oraz nie może wystąpić wymiana stron (tzn. $IRQL \geq DISPATCH_LEVEL$)



Deferred Procedure Calls (DPCs)

- Odroczone wywołanie procedury (DPC) jest wykorzystane do wykonania kodu przy przejściu z wyższego poziomu IRQL na niższy
 - Sterownik (proces jądra) kolejkuje żądania
 - Wykonanie zadań z kolejki następuje w momencie zakończenia pracy wszystkich przerw o wyższym IRQL
- Wykorzystane jest do generowania timeout'ów oraz przełączania wątków

